

21



Architecture Evaluation

Fear cannot be banished, but it can be calm and without panic; it can be mitigated by reason and evaluation.

—Vannevar Bush

We discussed analysis techniques in Chapter 14. Analysis lies at the heart of architecture evaluation, which is the process of determining if an architecture is fit for the purpose for which it is intended. Architecture is such an important contributor to the success of a system and software engineering project that it makes sense to pause and make sure that the architecture you’ve designed will be able to provide all that’s expected of it. That’s the role of evaluation. Fortunately there are mature methods to evaluate architectures that use many of the concepts and techniques you’ve already learned in previous chapters of this book.

21.1 Evaluation Factors

Evaluation usually takes one of three forms:

- Evaluation by the designer within the design process
- Evaluation by peers within the design process
- Analysis by outsiders once the architecture has been designed

Evaluation by the Designer

Every time the designer makes a key design decision or completes a design milestone, the chosen and competing alternatives should be evaluated using the analysis techniques of Chapter 14. Evaluation by the designer is the “test” part of the “generate-and-test” approach to architecture design that we discussed in Chapter 17.

How much analysis? This depends on the importance of the decision. Obviously, decisions made to achieve one of the driving architectural requirements should be subject to more analysis than others, because these are the ones that will shape critical portions of the architecture. But in all cases, performing analysis is a matter of cost and benefit. Do not spend more time on a decision than it is worth, but also do not spend less time on an important decision than it needs. Some specific considerations include these:

- *The importance of the decision.* The more important the decision, the more care should be taken in making it and making sure it's right.
- *The number of potential alternatives.* The more alternatives, the more time could be spent in evaluating them. Try to eliminate alternatives quickly so that the number of viable potential alternatives is small.
- *Good enough as opposed to perfect.* Many times, two possible alternatives do not differ dramatically in their consequences. In such a case, it is more important to make a choice and move on with the design process than it is to be absolutely certain that the best choice is being made. Again, do not spend more time on a decision than it is worth.

Peer Review

Architectural designs can be peer reviewed just as code can be peer reviewed. A peer review can be carried out at any point of the design process where a candidate architecture, or at least a coherent reviewable part of one, exists. There should be a fixed amount of time allocated for the peer review, at least several hours and possibly half a day. A peer review has several steps:

1. *The reviewers determine a number of quality attribute scenarios to drive the review.* Most of the time these scenarios will be architecturally significant requirements, but they need not be. These scenarios can be developed by the review team or by additional stakeholders.
2. *The architect presents the portion of the architecture to be evaluated.* (At this point, comprehensive documentation for it may not exist.) The reviewers individually ensure that they understand the architecture. Questions at this point are specifically for understanding. There is no debate about the decisions that were made. These come in the next step.
3. *For each scenario, the designer walks through the architecture and explains how the scenario is satisfied.* (If the architecture is already documented, then the reviews can use it to assess for themselves how it satisfies the scenario.) The reviewers ask questions to determine two different types of information. First, they want to determine that the scenario is, in fact, satisfied. Second, they want to determine whether any of the other scenarios being considered will not be satisfied because of the decisions made in the portion of the architecture being reviewed.

4. *Potential problems are captured.* The list of potential problems forms the basis for the follow-up of the review. If the potential problem is a real problem, then it either must be fixed or a decision must be explicitly made by the designers and the project manager that they are willing to accept the problem and its probability of occurrence.

If the designers are using the ADD process described in Chapter 17, then a peer review can be done at the end of step 3 of each ADD iteration.

Analysis by Outsiders

Outside evaluators can cast an objective eye on an architecture. “Outside” is relative; this may mean outside the development project, outside the business unit where the project resides but within the same company; or outside the company altogether. To the degree that evaluators are “outside,” they are less likely to be afraid to bring up sensitive problems, or problems that aren’t apparent because of organizational culture or because “we’ve always done it that way.”

Often, outsiders are chosen because they possess specialized knowledge or experience, such as knowledge about a quality attribute that’s important to the system being examined, or long experience in successfully evaluating architectures.

Also, whether justified or not, managers tend to be more inclined to listen to problems uncovered by an outside team hired at considerable cost. (This can be understandably frustrating to project staff who may have been complaining about the same problems to no avail for months.)

In principle, an outside team may evaluate a completed architecture, an incomplete architecture, or a portion of an architecture. In practice, because engaging them is complicated and often expensive, they tend to be used to evaluate complete architectures.

Contextual Factors

For peer reviews or outside analysis, there are a number of contextual factors that must be considered when structuring an evaluation. These include the artifacts available, whether the results are public or private, the number and skill of evaluators, the number and identity of the participating stakeholders, and how the business goals are understood by the evaluators.

- *What artifacts are available?* To perform an architectural evaluation, there must be an artifact that describes the architecture. This must be located and made available. Some evaluations may take place after the system is operational. In this case, recovery tools as described in Chapter 20 may be used both to assist in discovering the architecture and to test that the as-built system conforms to the as-designed system.

- *Who sees the results?* Some evaluations are performed with the full knowledge and participation of all of the stakeholders. Others are performed more privately. The private evaluations may be done for a variety of reasons, ranging from corporate culture to (in one case we know about) an executive wanting to determine which of a collection of competitive systems he should back in an internal dispute about the systems.
- *Who performs the evaluation?* Evaluations can be carried out by an individual or a team. In either case, the evaluator(s) should be highly skilled in the domain and the various quality attributes for which the system is to be evaluated. And for carrying out evaluation methods with extensive stakeholder involvement, excellent organizational and facilitation skills are a must.
- *Which stakeholders will participate?* The evaluation process should provide a method to elicit the goals and concerns that the important stakeholders have regarding the system. Identifying the individuals who are needed and assuring their participation in the evaluation is critical.
- *What are the business goals?* The evaluation should answer whether the system will satisfy the business goals. If the business goals are not explicitly captured and prioritized prior to the evaluation, then there should be a portion of the evaluation dedicated to doing so.

21.2 The Architecture Tradeoff Analysis Method

The Architecture Tradeoff Analysis Method (ATAM) has been used for over a decade to evaluate software architectures in domains ranging from automotive to financial to defense. The ATAM is designed so that evaluators need not be familiar with the architecture or its business goals, the system need not yet be constructed, and there may be a large number of stakeholders.

Participants in the ATAM

The ATAM requires the participation and mutual cooperation of three groups:

- *The evaluation team.* This group is external to the project whose architecture is being evaluated. It usually consists of three to five people. Each member of the team is assigned a number of specific roles to play during the evaluation. (See Table 21.1 for a description of these roles, along with a set of desirable characteristics for each. A single person may adopt several roles in an ATAM.) The evaluation team may be a standing unit in which architecture evaluations are regularly performed, or its members may be chosen from a pool of architecturally savvy individuals for the occasion.

They may work for the same organization as the development team whose architecture is on the table, or they may be outside consultants. In any case, they need to be recognized as competent, unbiased outsiders with no hidden agendas or axes to grind.

- *Project decision makers.* These people are empowered to speak for the development project or have the authority to mandate changes to it. They usually include the project manager, and if there is an identifiable customer who is footing the bill for the development, he or she may be present (or represented) as well. The architect is always included—a cardinal rule of architecture evaluation is that the architect must willingly participate.
- *Architecture stakeholders.* Stakeholders have a vested interest in the architecture performing as advertised. They are the ones whose ability to do their job hinges on the architecture promoting modifiability, security, high reliability, or the like. Stakeholders include developers, testers, integrators, maintainers, performance engineers, users, builders of systems interacting with the one under consideration, and others listed in Chapter 3. Their job during an evaluation is to articulate the specific quality attribute goals that the architecture should meet in order for the system to be considered a success. A rule of thumb—and that is all it is—is that you should expect to enlist 12 to 15 stakeholders for the evaluation of a large enterprise-critical architecture. Unlike the evaluation team and the project decision makers, stakeholders do not participate in the entire exercise.

TABLE 21.1 ATAM Evaluation Team Roles

Role	Responsibilities
Team Leader	Sets up the evaluation; coordinates with client, making sure client's needs are met; establishes evaluation contract; forms evaluation team; sees that final report is produced and delivered (although the writing may be delegated)
Evaluation Leader	Runs evaluation; facilitates elicitation of scenarios; administers scenario selection/prioritization process; facilitates evaluation of scenarios against architecture; facilitates on-site analysis
Scenario Scribe	Writes scenarios on flipchart or whiteboard during scenario elicitation; captures agreed-on wording of each scenario, halting discussion until exact wording is captured
Proceedings Scribe	Captures proceedings in electronic form on laptop or workstation: raw scenarios, issue(s) that motivate each scenario (often lost in the wording of the scenario itself), and resolution of each scenario when applied to architecture(s); also generates a printed list of adopted scenarios for handout to all participants
Questioner	Raises issues of architectural interest, usually related to the quality attributes in which he or she has expertise

Outputs of the ATAM

As in any testing process, a large benefit derives from preparing for the test. In preparation for an ATAM exercise, the project's decision makers must prepare the following:

1. *A concise presentation of the architecture.* One of the requirements of the ATAM is that the architecture be presented in one hour, which leads to an architectural presentation that is both concise and, usually, understandable.
2. *Articulation of the business goals.* Frequently, the business goals presented in the ATAM are being seen by some of the assembled participants for the first time, and these are captured in the outputs. This description of the business goals survives the evaluation and becomes part of the project's legacy.

The ATAM uses prioritized quality attribute scenarios as the basis for evaluating the architecture, and if those scenarios do not already exist (perhaps as a result of a prior requirements capture exercise or ADD activity), they are generated by the participants as part of the ATAM exercise. Many times, ATAM participants have told us that one of the most valuable outputs of ATAM is this next output:

3. *Prioritized quality attribute requirements expressed as quality attribute scenarios.* These quality attribute scenarios take the form described in Chapter 4. These also survive past the evaluation and can be used to guide the architecture's evolution.

The primary output of the ATAM is a set of issues of concern about the architecture. We call these risks:

4. *A set of risks and nonrisks.* A risk is defined in the ATAM as an architectural decision that may lead to undesirable consequences in light of stated quality attribute requirements. Similarly, a nonrisk is an architectural decision that, upon analysis, is deemed safe. The identified risks form the basis for an architectural risk mitigation plan.
5. *A set of risk themes.* When the analysis is complete, the evaluation team examines the full set of discovered risks to look for overarching themes that identify systemic weaknesses in the architecture or even in the architecture process and team. If left untreated, these risk themes will threaten the project's business goals.

Finally, along the way, other information about the architecture is discovered and captured:

6. *Mapping of architectural decisions to quality requirements.* Architectural decisions can be interpreted in terms of the qualities that they support or hinder. For each quality attribute scenario examined during an ATAM, those

architectural decisions that help to achieve it are determined and captured. This can serve as a statement of rationale for those decisions.

7. *A set of identified sensitivity and tradeoff points.* These are architectural decisions that have a marked effect on one or more quality attributes.

The outputs of the ATAM are used to build a final written report that recaps the method, summarizes the proceedings, captures the scenarios and their analysis, and catalogs the findings.

There are intangible results of an ATAM-based evaluation. These include a palpable sense of community on the part of the stakeholders, open communication channels between the architect and the stakeholders, and a better overall understanding on the part of all participants of the architecture and its strengths and weaknesses. While these results are hard to measure, they are no less important than the others and often are the longest-lasting.

Phases of the ATAM

Activities in an ATAM-based evaluation are spread out over four phases:

- In phase 0, “Partnership and Preparation,” the evaluation team leadership and the key project decision makers informally meet to work out the details of the exercise. The project representatives brief the evaluators about the project so that the team can be supplemented by people who possess the appropriate expertise. Together, the two groups agree on logistics, such as the time and place of meetings, who brings the flipcharts, and who supplies the donuts and coffee. They also agree on a preliminary list of stakeholders (by name, not just role), and they negotiate on when the final report is to be delivered and to whom. They deal with formalities such as a statement of work or nondisclosure agreements. The evaluation team examines the architecture documentation to gain an understanding of the architecture and the major design approaches that it comprises. Finally, the evaluation team leader explains what information the manager and architect will be expected to show during phase 1, and helps them construct their presentations if necessary.
- Phase 1 and phase 2 are the evaluation phases, where everyone gets down to the business of analysis. By now the evaluation team will have studied the architecture documentation and will have a good idea of what the system is about, the overall architectural approaches taken, and the quality attributes that are of paramount importance. During phase 1, the evaluation team meets with the project decision makers (for one to two days) to begin information gathering and analysis. For phase 2, the architecture’s stakeholders join the proceedings and analysis continues, typically for two days. Unlike the other phases, phase 1 and phase 2 comprise a set of specific steps; these are detailed in the next section.

TABLE 21.2 ATAM Phases and Their Characteristics

Phase	Activity	Participants	Typical Duration
0	Partnership and preparation	Evaluation team leadership and key project decision makers	Proceeds informally as required, perhaps over a few weeks
1	Evaluation	Evaluation team and project decision makers	1–2 days followed by a hiatus of 1–3 weeks
2	Evaluation (continued)	Evaluation team, project decision makers, and stakeholders	2 days
3	Follow-up	Evaluation team and evaluation client	1 week

Source: Adapted from [Clements 01b].

- Phase 3 is follow-up, in which the evaluation team produces and delivers a written final report. It is first circulated to key stakeholders to make sure that it contains no errors of understanding, and after this review is complete it is delivered to the person who commissioned the evaluation.

Table 21.2 shows the four phases of the ATAM, who participates in each one, and an approximate timetable.

Steps of the Evaluation Phases

The ATAM analysis phases (phase 1 and phase 2) consist of nine steps. Steps 1 through 6 are carried out in phase 1 with the evaluation team and the project’s decision makers: typically, the architecture team, project manager, and project sponsor. In phase 2, with all stakeholders present, steps 1 through 6 are summarized and steps 7 through 9 are carried out.

Table 21.3 shows a typical agenda for the first day of phase 1, which covers steps 1 through 5. Step 6 in phase 1 is carried out the next day.

Step 1: Present the ATAM. The first step calls for the evaluation leader to present the ATAM to the assembled project representatives. This time is used to explain the process that everyone will be following, to answer questions, and to set the context and expectations for the remainder of the activities. Using a standard presentation, the leader describes the ATAM steps in brief and the outputs of the evaluation.

Step 2: Present the Business Drivers. Everyone involved in the evaluation—the project representatives as well as the evaluation team members—needs to understand the context for the system and the primary business drivers

motivating its development. In this step, a project decision maker (ideally the project manager or the system's customer) presents a system overview from a business perspective. The presentation should describe the following:

- The system's most important functions
- Any relevant technical, managerial, economic, or political constraints
- The business goals and context as they relate to the project
- The major stakeholders
- The architectural drivers (that is, the architecturally significant requirements)

Step 3: Present the Architecture. Here, the lead architect (or architecture team) makes a presentation describing the architecture at an appropriate level of detail. The “appropriate level” depends on several factors: how much of the architecture has been designed and documented; how much time is available; and the nature of the behavioral and quality requirements.

In this presentation the architect covers technical constraints such as operating system, hardware, or middleware prescribed for use, and other systems with which the system must interact. Most important, the architect describes the architectural approaches (or patterns, or tactics, if the architect is fluent in that vocabulary) used to meet the requirements.

To make the most of limited time, the architect's presentation should have a high signal-to-noise ratio. That is, it should convey the essence of the architecture and not stray into ancillary areas or delve too deeply into the details of just a few aspects. Thus, it is extremely helpful to brief the architect beforehand (in phase 0) about the information the evaluation team requires. A template such as the one in the sidebar can help the architect prepare the presentation. Depending on the architect, a dress rehearsal can be included as part of the phase 0 activities.

TABLE 21.3 Agenda for Day 1 of the ATAM

Time	Activity
0830 – 1000	Introductions; Step 1: Present the ATAM
1000 – 1100	Step 2: Present Business Drivers
1100 – 1130	Break
1130 – 1230	Step 3: Present Architecture
1230 – 1330	Lunch
1330 – 1430	Step 4: Identify Architectural Approaches
1430 – 1530	Step 5: Generate Utility Tree
1530 – 1600	Break
1600 – 1700	Step 5: Generate Utility Tree (continued)

Architecture Presentation (Approximately 20 slides; 60 Minutes)

Driving architectural requirements, the measurable quantities you associate with these requirements, and any existing standards/models/approaches for meeting these (2–3 slides)

Important architectural information (4–8 slides):

- Context diagram—the system within the context in which it will exist. Humans or other systems with which the system will interact.
- Module or layer view—the modules (which may be subsystems or layers) that describe the system's decomposition of functionality, along with the objects, procedures, functions that populate these, and the relations among them (e.g., procedure call, method invocation, callback, containment).
- Component-and-connector view—processes, threads along with the synchronization, data flow, and events that connect them.
- Deployment view—CPUs, storage, external devices/sensors along with the networks and communication devices that connect them. Also shown are the processes that execute on the various processors.

Architectural approaches, patterns, or tactics employed, including what quality attributes they address and a description of how the approaches address those attributes (3–6 slides):

- Use of commercial off-the-shelf (COTS) products and how they are chosen/integrated (1–2 slides).
- Trace of 1 to 3 of the most important use case scenarios. If possible, include the runtime resources consumed for each scenario (1–3 slides).
- Trace of 1 to 3 of the most important change scenarios. If possible, describe the change impact (estimated size/difficulty of the change) in terms of the changed modules or interfaces (1–3 slides).
- Architectural issues/risks with respect to meeting the driving architectural requirements (2–3 slides).
- Glossary (1 slide).

Source: Adapted from [Clements 01b].

As may be seen in the presentation template, we expect architectural views, as described in Chapters 1 and 18, to be the primary vehicle for the architect to convey the architecture. Context diagrams, component-and-connector views, module decomposition or layered views, and the deployment view are useful in almost every evaluation, and the architect should be prepared to show them. Other views can be presented if they contain information relevant to the architecture at hand, especially information relevant to achieving important quality attribute goals.

As a rule of thumb, the architect should present the views that he or she found most important during the creation of the architecture and the views that help to reason about the most important quality attribute concerns of the system.

During the presentation, the evaluation team asks for clarification based on their phase 0 examination of the architecture documentation and their knowledge of the business drivers from the previous step. They also listen for and write down any architectural tactics or patterns they see employed.

Step 4: Identify Architectural Approaches. The ATAM focuses on analyzing an architecture by understanding its architectural approaches. As we saw in Chapter 13, architectural patterns and tactics are useful for (among other reasons) the known ways in which each one affects particular quality attributes. A layered pattern tends to bring portability and maintainability to a system, possibly at the expense of performance. A publish-subscribe pattern is scalable in the number of producers and consumers of data. The active redundancy tactic promotes high availability. And so forth.

By now, the evaluation team will have a good idea of what patterns and tactics the architect used in designing the system. They will have studied the architecture documentation, and they will have heard the architect's presentation in step 3. During that step, the architect is asked to explicitly name the patterns and tactics used, but the team should also be adept at spotting ones not mentioned.

In this short step, the evaluation team simply catalogs the patterns and tactics that have been identified. The list is publicly captured by the scribe for all to see and will serve as the basis for later analysis.

Step 5: Generate Quality Attribute Utility Tree. In this step, the quality attribute goals are articulated in detail via a quality attribute utility tree. Utility trees, which were described in Chapter 16, serve to make the requirements concrete by defining precisely the relevant quality attribute requirements that the architects were working to provide.

The important quality attribute goals for the architecture under consideration were named in step 2, when the business drivers were presented, but not to any degree of specificity that would permit analysis. Broad goals such as “modifiability” or “high throughput” or “ability to be ported to a number of platforms” establish important context and direction, and provide a backdrop against which subsequent information is presented. However, they are not specific enough to let us tell if the architecture suffices. Modifiable in what way? Throughput that is how high? Ported to what platforms and in how much time?

In this step, the evaluation team works with the project decision makers to identify, prioritize, and refine the system's most important quality attribute goals. These are expressed as scenarios, as described in Chapter 4, which populate the leaves of the utility tree.

Step 6: Analyze Architectural Approaches. Here the evaluation team examines the highest-ranked scenarios (as identified in the utility tree) one at a time; the architect is asked to explain how the architecture supports each one. Evaluation team members—especially the questioners—probe for the architectural approaches that the architect used to carry out the scenario. Along the way, the evaluation team documents the relevant architectural decisions and identifies and catalogs their risks, nonrisks, sensitivity points, and tradeoffs. For well-known approaches, the evaluation team asks how the architect overcame known weaknesses in the approach or how the architect gained assurance that the approach sufficed. The goal is for the evaluation team to be convinced that the instantiation of the approach is appropriate for meeting the attribute-specific requirements for which it is intended.

Scenario walkthrough leads to a discussion of possible risks, nonrisks, sensitivity points, or tradeoff points. For example:

- The frequency of heartbeats affects the time in which the system can detect a failed component. Some assignments will result in unacceptable values of this response—these are risks.
- The number of simultaneous database clients will affect the number of transactions that a database can process per second. Thus, the assignment of clients to the server is a sensitivity point with respect to the response as measured in transactions per second.
- The frequency of heartbeats determines the time for detection of a fault. Higher frequency leads to improved availability but will also consume more processing time and communication bandwidth (potentially leading to reduced performance). This is a tradeoff.

These, in turn, may catalyze a deeper analysis, depending on how the architect responds. For example, if the architect cannot characterize the number of clients and cannot say how load balancing will be achieved by allocating processes to hardware, there is little point in a sophisticated performance analysis. If such questions can be answered, the evaluation team can perform at least a rudimentary, or back-of-the-envelope, analysis to determine if these architectural decisions are problematic vis-à-vis the quality attribute requirements they are meant to address.

The analysis is not meant to be comprehensive. The key is to elicit sufficient architectural information to establish some link between the architectural decisions that have been made and the quality attribute requirements that need to be satisfied.

Figure 21.1 shows a template for capturing the analysis of an architectural approach for a scenario. As shown, based on the results of this step, the evaluation team can identify and record a set of sensitivity points and tradeoffs, risks, and nonrisks.

At the end of step 6, the evaluation team should have a clear picture of the most important aspects of the entire architecture, the rationale for key design decisions, and a list of risks, nonrisks, sensitivity points, and tradeoff points.

At this point, phase 1 is concluded.

Scenario #: A12		Scenario: Detect and recover from HW failure of main switch.			
Attribute(s)	Availability				
Environment	Normal operations				
Stimulus	One of the CPUs fails				
Response	0.999999 availability of switch				
Architectural decisions		Sensitivity	Tradeoff	Risk	Nonrisk
Backup CPU(s)		S2		R8	
No backup data channel		S3	T3	R9	
Watchdog		S4			N12
Heartbeat		S5			N13
Failover routing		S6			N14
Reasoning	Ensures no common mode failure by using different hardware and operating system (see Risk 8) Worst-case rollover is accomplished in 4 seconds as computing state takes that long at worst Guaranteed to detect failure within 2 seconds based on rates of heartbeat and watchdog Watchdog is simple and has proved reliable Availability requirement might be at risk due to lack of backup data channel ... (see Risk 9)				
Architecture diagram	<pre>graph LR; In(()) --> P[Primary CPU OS1]; In --> B[Backup CPU with Watchdog OS2]; P -.-> heartbeat 1 sec. B; P --> S[Switch CPU OS1]; B --> S; S --> Out(())</pre>				

FIGURE 21.1 Example of architecture approach analysis (adapted from [Clements 01b])

Hiatus and Start of Phase 2. The evaluation team summarizes what it has learned and interacts informally (usually by phone) with the architect during a hiatus of a week or two. More scenarios might be analyzed during this period, if desired, or questions of clarification can be resolved.

Phase 2 is attended by an expanded list of participants with additional stakeholders attending. To use an analogy from programming: Phase 1 is akin to when you test your own program, using your own criteria. Phase 2 is when you give your program to an independent quality assurance group, who will likely subject your program to a wider variety of tests and environments.

In phase 2, step 1 is repeated so that the stakeholders understand the method and the roles they are to play. Then the evaluation leader recaps the results of steps 2 through 6, and shares the current list of risks, nonrisks, sensitivity points, and tradeoffs. Now the stakeholders are up to speed with the evaluation results so far, and the remaining three steps can be carried out.

Step 7: Brainstorm and Prioritize Scenarios. In this step, the evaluation team asks the stakeholders to brainstorm scenarios that are operationally meaningful with respect to the stakeholders' individual roles. A maintainer will likely propose a modifiability scenario, while a user will probably come up with a scenario that expresses useful functionality or ease of operation, and a quality assurance person will propose a scenario about testing the system or being able to replicate the state of the system leading up to a fault.

While utility tree generation (step 5) is used primarily to understand how the architect perceived and handled quality attribute architectural drivers, the purpose of scenario brainstorming is to take the pulse of the larger stakeholder community: to understand what system success means for them. Scenario brainstorming works well in larger groups, creating an atmosphere in which the ideas and thoughts of one person stimulate others' ideas.

Once the scenarios have been collected, they must be prioritized, for the same reasons that the scenarios in the utility tree needed to be prioritized: the evaluation team needs to know where to devote its limited analytical time. First, stakeholders are asked to merge scenarios they feel represent the same behavior or quality concern. Then they vote for those they feel are most important. Each stakeholder is allocated a number of votes equal to 30 percent of the number of scenarios,¹ rounded up. So, if there were 40 scenarios collected, each stakeholder would be given 12 votes. These votes can be allocated in any way that the stakeholder sees fit: all 12 votes for 1 scenario, 1 vote for each of 12 distinct scenarios, or anything in between.

The list of prioritized scenarios is compared with those from the utility tree exercise. If they agree, it indicates good alignment between what the architect had in mind and what the stakeholders actually wanted. If additional driving scenarios are discovered—and they usually are—this may itself be a risk, if the discrepancy is large. This would indicate that there was some disagreement in the system's important goals between the stakeholders and the architect.

1. This is a common facilitated brainstorming technique.

Step 8: Analyze Architectural Approaches. After the scenarios have been collected and prioritized in step 7, the evaluation team guides the architect in the process of carrying out the highest ranked scenarios. The architect explains how relevant architectural decisions contribute to realizing each one. Ideally this activity will be dominated by the architect's explanation of scenarios in terms of previously discussed architectural approaches.

In this step the evaluation team performs the same activities as in step 6, using the highest-ranked, newly generated scenarios.

Typically, this step might cover the top five to ten scenarios, as time permits.

Step 9: Present Results. In step 9, the evaluation team groups risks into risk themes, based on some common underlying concern or systemic deficiency. For example, a group of risks about inadequate or out-of-date documentation might be grouped into a risk theme stating that documentation is given insufficient consideration. A group of risks about the system's inability to function in the face of various hardware and/or software failures might lead to a risk theme about insufficient attention to backup capability or providing high availability.

For each risk theme, the evaluation team identifies which of the business drivers listed in step 2 are affected. Identifying risk themes and then relating them to specific drivers brings the evaluation full circle by relating the final results to the initial presentation, thus providing a satisfying closure to the exercise. As important, it elevates the risks that were uncovered to the attention of management. What might otherwise have seemed to a manager like an esoteric technical issue is now identified unambiguously as a threat to something the manager is on record as caring about.

The collected information from the evaluation is summarized and presented to stakeholders. This takes the form of a verbal presentation with slides. The evaluation leader recapitulates the steps of the ATAM and all the information collected in the steps of the method, including the business context, driving requirements, constraints, and architecture. Then the following outputs are presented:

- The architectural approaches documented
- The set of scenarios and their prioritization from the brainstorming
- The utility tree
- The risks discovered
- The nonrisks documented
- The sensitivity points and tradeoff points found
- Risk themes and the business drivers threatened by each one

“ . . . but it was OK.”

Years of experience have taught us that no architecture evaluation exercise ever goes completely by the book. And yet for all the ways that an exercise might go terribly wrong, for all the details that can be overlooked, for all the fragile egos that can be bruised, and for all the high stakes that are on the table, we have never had an architecture evaluation exercise spiral out of control. Every single one has been a success, as measured by the feedback we gather from clients.

While they all turned out successfully, there were a few memorable cliffhangers.

More than once, we began an architecture evaluation only to discover that the development organization had no architecture to be evaluated. Sometimes there was a stack of class diagrams or vague text descriptions masquerading as an architecture. Once we were promised that the architecture would be ready by the time the exercise began, but in spite of good intentions, it wasn't. (We weren't always so prudent about pre-exercise preparation and qualification. Our current diligence was a result of experiences like these.) But it was OK. In cases like these, the evaluation's main results included the articulated set of quality attributes, a “whiteboard” architecture sketched during the exercise, plus a set of documentation obligations on the architect. In all cases, the client felt that the detailed scenarios, the analysis we were able to perform on the elicited architecture, plus the recognition of what needed to be done, more than justified the exercise.

A couple of times we began an evaluation only to lose the architect in the middle of the exercise. In one case, the architect resigned between preparation and execution of the evaluation. This was an organization in turmoil and the architect simply got a better offer in a calmer environment elsewhere. Normally we don't proceed without the architect, but it was OK. In this case the architect's apprentice stepped in. A little additional prework to prepare him, and we were all set. The evaluation went off as planned, and the preparation that the apprentice did for the exercise helped mightily to prepare him to step into the architect's shoes.

Once we discovered halfway through an ATAM exercise that the architecture we had prepared to evaluate was being jettisoned in favor of a new one that nobody had bothered to mention. During step 6 of phase 1, the architect responded to a problem raised by a scenario by casually mentioning that “the new architecture” would not suffer from that deficiency. Everyone in the room, stakeholders and evaluators alike, looked at each other in the puzzled silence that followed. “What new architecture?” I asked blankly, and out it came. The developing organization (a contractor for the U.S. military, which had commissioned the evaluation), had prepared a new architecture for the system, to handle the more stringent requirements they knew were coming in the future. We called a timeout, conferred with the architect and the client, and decided to continue the exercise using the new architecture as the subject instead of the old. We backed up to step 3 (the architecture presentation), but everything else on the table—business drivers, utility

tree, scenarios—still were completely valid. The evaluation proceeded as before, and at the conclusion of the exercise our military client was extremely pleased at the knowledge gained.

In perhaps the most bizarre evaluation in our experience, we lost the architect midway through phase 2. The client for this exercise was the project manager in an organization undergoing a massive restructuring. The manager was a pleasant gentleman with a quick sense of humor, but there was an undercurrent about him that said he was not to be crossed. The architect was being reassigned to a different part of the organization in the near future; this was tantamount to being fired from the project, and the manager said he wanted to establish the quality of the architecture before his architect's awkward departure. (We didn't find any of this out until after the evaluation.) When we set up the ATAM exercise, the manager suggested that the junior designers attend. "They might learn something," he said. We agreed. As the exercise began, our schedule (which was very tight to begin with) kept being disrupted. The manager wanted us to meet with his company's executives. Then he wanted us to have a long lunch with someone who could, he said, give us more architectural insights. The executives, it turned out, were busy just now, and so could we come back and meet with them a bit later? By now, phase 2 was thrown off schedule by so much that the architect, to our horror, had to leave to fly back to his home in a distant city. He was none too happy that his architecture was going to be evaluated without him. The junior designers, he said, would never be able to answer our questions. Before his departure, our team huddled. The exercise seemed to be teetering on the brink of disaster. We had an unhappy departing architect, a blown schedule, and questionable expertise available. We decided to split our evaluation team. One half of the team would continue with phase 2 using the junior designers as our information resource. The second half of the team would continue with phase 2 by telephone the next day with the architect. Somehow we would make the best of a bad situation.

Surprisingly, the project manager seemed completely unperturbed by the turn of events. "It will work out, I'm sure," he said pleasantly, and then retreated to confer with various vice presidents about the reorganization.

I led the team interviewing the junior designers. We had never gotten a completely satisfactory architecture presentation from the architect. Discrepancies in the documentation were met with a breezy "Oh, well, that's not how it really works." So I decided to start over with ATAM step 3. We asked the half dozen or so designers what their view of the architecture was. "Could you draw it?" I asked them. They looked at each other nervously, but one said, "I think I can draw part of it." He took to the whiteboard and drew a very reasonable component-and-connector view. Someone else volunteered to draw a process view. A third person drew the architecture for an important offline part of the system. Others jumped in to assist.

As we looked around the room, everyone was busy transcribing the whiteboard pictures. None of the pictures corresponded to anything we had seen in the documentation so far. "Are these diagrams documented

anywhere?" I asked. One of the designers looked up from his busy scribbling for a moment to grin. "They are now," he said.

As we proceeded to step 8, analyzing the architecture using the scenarios previously captured, the designers did an astonishingly good job of working together to answer our questions. Nobody knew everything, but everybody knew something. Together in a half day, they produced a clear and consistent picture of the whole architecture that was much more coherent and understandable than anything the architect had been willing to produce in two whole days of pre-exercise discussion. And by the end of phase 2, the design team was transformed. This erstwhile group of information-starved individuals with limited compartmentalized knowledge became a true architecture team. The members drew out and recognized each others' expertise. This expertise was revealed and validated in front of everyone—and most important, in front of their project manager, who had slipped back into the room to observe. There was a look of supreme satisfaction on his face. It began to dawn on me that—you guessed it—it was OK.

It turned out that this project manager knew how to manipulate events and people in ways that would have impressed Machiavelli. The architect's departure was not because of the reorganization, but merely coincident with it. The project manager had orchestrated it. The architect had, the manager felt, become too autocratic and dictatorial, and the manager wanted the junior design staff to be given the opportunity to mature and contribute. The architect's mid-exercise departure was exactly what the project manager had wanted. And the design team's emergence under fire had been the primary purpose of the evaluation exercise all along. Although we found several important issues related to the architecture, the project manager knew about every one of them before we ever arrived. In fact, he made sure we uncovered some of them by a few discreet remarks during breaks or after a day's session.

Was this exercise a success? The client could not have been more pleased. His instincts about the architecture's strengths and weaknesses were confirmed. We were instrumental in helping his design team, which would guide the system through the stormy seas of the company's reorganization, come together as an effective and cohesive unit at exactly the right time. And the client was so pleased with our final report that he made sure the company's board of directors saw it.

These cliffhangers certainly stand out in our memory. There was no architecture documented. But it was OK. It wasn't the right architecture. But it was OK. There was no architect. But it was OK. The client really only wanted to effect a team reorganization. In every instance we reacted as reasonably as we could, and each time it was OK.

Why? Why, time after time, does it turn out OK? I think there are three reasons.

First, the people who have commissioned the architecture evaluation really want it to succeed. The architect, developers, and stakeholders

assembled at the client's behest also want it to succeed. As a group, they help to keep the exercise marching toward the goal of architectural insight. Second, we are always honest. If we feel that the exercise is derailing, we call a timeout and confer among ourselves, and usually confer with the client. While a small amount of bravado can come in handy during an exercise, we never, ever try to bluff our way through an evaluation. Participants can detect that instinctively, and the evaluation team must never lose the respect of the other participants. Third, the methods are constructed to establish and maintain a steady consensus throughout the exercise. There are no surprises at the end. The participants lay down the ground rules for what constitutes a suitable architecture, and they contribute to the risks uncovered at every step of the way.

So: Do the best job you can. Be honest. Trust the methods. Trust in the goodwill and good intentions of the people you have assembled. And it will be OK. (Adapted from [Clements 01b])

—PCC

21.3 Lightweight Architecture Evaluation

Although we attempt to use time in an ATAM exercise as efficiently as possible, it remains a substantial undertaking. It requires some 20 to 30 person-days of effort from an evaluation team, plus even more for the architect and stakeholders. Investing this amount of time only makes sense on a large and costly project, where the risks of making a major mistake in the architecture are unacceptable.

For this reason, we have developed a Lightweight Architecture Evaluation method, based on the ATAM, for smaller, less risky projects. A Lightweight Architecture Evaluation exercise may take place in a single day, or even a half-day meeting. It may be carried out entirely by members internal to the organization. Of course this lower level of scrutiny and objectivity may not probe the architecture as deeply, but this is a cost/benefit tradeoff that is entirely appropriate for many projects.

Because the participants are all internal to the organization and fewer in number than for the ATAM, giving everyone their say and achieving a shared understanding takes much less time. Hence the steps and phases of a Lightweight Architecture Evaluation can be carried out more quickly. A suggested schedule for phases 1 and 2 is shown in Table 21.4.

TABLE 21.4 A Typical Agenda for Lightweight Architecture Evaluation

Step	Time Allotted	Notes
1: Present the ATAM	0 hrs	The participants are familiar with the process. This step may be omitted.
2: Present Business Drivers	0.25 hrs	The participants are expected to understand the system and its business goals and their priorities. Fifteen minutes is allocated for a brief review to ensure that these are fresh in everyone's mind and that there are no surprises.
3: Present Architecture	0.5 hrs	Again, all participants are expected to be familiar with the system and so a brief overview of the architecture, using at least module and C&C views, is presented and 1 to 2 scenarios are traced through these views.
4: Identify Architectural Approaches	0.25 hrs	The architecture approaches for specific quality attribute concerns are identified by the architect. This may be done as a portion of step 3.
5: Generate Quality Attribute Utility Tree	Variable 0.5 hrs – 1.5 hrs	Scenarios might exist: part of previous evals, part of design, part of requirements elicitation. If you've got 'em, use 'em and make them into a tree. Half hour. Otherwise, it will take longer. A utility tree should already exist; the team reviews the existing tree and updates it, if needed, with new scenarios, new response goals, or new scenario priorities and risk assessments.
6: Analyze Architectural Approaches	2–3 hrs	This step—mapping the highly ranked scenarios onto the architecture—consumes the bulk of the time and can be expanded or contracted as needed.
7: Brainstorm and Prioritize Scenarios	0 hrs	This step can be omitted as the assembled (internal) stakeholders are expected to contribute scenarios expressing their concerns in step 5.
8: Analyze Architectural Approaches	0 hrs	This step is also omitted, since all analysis is done in step 6.
9: Present Results	0.5 hrs	At the end of an evaluation, the team reviews the existing and newly discovered risks, non-risks, sensitivities, and tradeoffs and discusses whether any new risk themes have arisen.
TOTAL	4–6 hrs	

There is no final report, but (as in the regular ATAM) a scribe is responsible for capturing results, which can then be distributed and serve as the basis for risk remediation.

An entire Lightweight Architecture Evaluation can be prosecuted in less than a day—perhaps an afternoon. The results will depend on how well the assembled team understands the goals of the method, the techniques of the method, and the system itself. The evaluation team, being internal, is typically *not* objective, and this may compromise the value of its results—one tends to hear fewer new ideas

and fewer dissenting opinions. But this version of evaluation is inexpensive, easy to convene, and relatively low ceremony, so it can be quickly deployed whenever a project wants an architecture quality assurance sanity check.

21.4 Summary

If a system is important enough for you to explicitly design its architecture, then that architecture should be evaluated.

The number of evaluations and the extent of each evaluation may vary from project to project. A designer should perform an evaluation during the process of making an important decision. Lightweight evaluations can be performed several times during a project as a peer review exercise.

The ATAM is a comprehensive method for evaluating software architectures. It works by having project decision makers and stakeholders articulate a precise list of quality attribute requirements (in the form of scenarios) and by illuminating the architectural decisions relevant to carrying out each high-priority scenario. The decisions can then be understood in terms of risks or nonrisks to find any trouble spots in the architecture.

Lightweight Architecture Evaluation, based on the ATAM, provides an inexpensive, low-ceremony architecture evaluation that can be carried out in an afternoon.

21.5 For Further Reading

For a more comprehensive treatment of the ATAM, see [Clements 01b].

Multiple case studies of applying the ATAM are available. They can be found by going to www.sei.cmu.edu/library and searching for “ATAM case study.”

To understand the historical roots of the ATAM, and to see a second (simpler) architecture evaluation method, you can read about the software architecture analysis method (SAAM) in [Kazman 94].

Several lighter weight architecture evaluation methods have been developed. They can be found in [Bouwers 10], [Kanwal 10], and [Bachmann 11].

Maranzano et al. have published a paper dealing with a long tradition of architecture evaluation at AT&T and its successor companies [Maranzano 05].

21.6 Discussion Questions

1. Think of a software system that you're working on. Prepare a 30-minute presentation on the business drivers for this system.
2. If you were going to evaluate the architecture for this system, who would you want to participate? What would be the stakeholder roles and who could you get to represent those roles?
3. Use the utility tree that you wrote for the ATM in Chapter 16 and the design that you sketched for the ATM in Chapter 17 to perform the scenario analysis step of the ATAM. Capture any risks and nonrisks that you discover. Better yet, perform the analysis on the design carried out by a colleague.
4. It is not uncommon for an organization to evaluate two competing architectures. How would you modify the ATAM to produce a quantitative output that facilitates this comparison?
5. Suppose you've been asked to evaluate the architecture for a system in confidence. The architect isn't available. You aren't allowed to discuss the evaluation with any of the system's stakeholders. How would you proceed?
6. Under what circumstances would you want to employ a full-strength ATAM and under what circumstances would you want to employ a Lightweight Architecture Evaluation?